ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ №3

Дисципліна: Функціональне програмування

Тема:	Нездатні до змін структури даних (immutability) у Python
Варіант:	№12 (Ресторанне меню: Dish, Ingredient, allergens, meta)
Студент:	Малецький Маркіян Володимирович
Група:	ФЕП-23с

1. Мета роботи

Дослідити принципи та практичне застосування незмінних структур даних (immutability) у Python. Опанувати заміну мутабельних in-place операцій на чисті функції оновлення (pure updates). Реалізувати незмінну модель предметної області за варіантом №12 з використанням tuple, frozenset, @dataclass(frozen=True, slots=True) та MappingProxyType, а також перевірити роботу тестів і швидкість виконання.

2. Архітектура та програмна реалізація

Для реалізації предметної області було створено наступні модулі:
• model.py: оголошено сутність Ingredient (назва, собівартість, вага) з валідацією полів та підтримкою хешування. Сутність Dish інкапсулює список інгредієнтів як tuple (замість list), набір алергенів як frozenset (для швидкої O(1) перевірки та операцій множин) і метадані у MappingProxyType (тільки для читання).
• core.py: реалізовано pure update функції (add_ingredient, add_allergen, update_price, apply_discount, update_meta), що повертають нові копії через dataclasses.replace. Також реалізовано утиліту freeze(obj) для глибокого заморожування вкладених структур та кешований розрахунок маржинальності через @lru_cache.

3. Результати тестування та валідація

Написано набір автоматизованих тестів у фреймворку pytest (файли test_immutability.py та test_performance.py):
• Незмінність і відсутність мутацій: перевірено, що після додавання інгредієнтів чи зміни цін вихідний об'єкт залишається без змін (updated is not original).
• Захист від запису: пряма спроба зміни полів викликає FrozenInstanceError, а спроба мутації словника метаданих викликає TypeError.
• Хешованість: об'єкти коректно додаються до множин та використовуються як ключі.
• Продуктивність: підтверджено ефективність мемоізації чистої функції calculate_margin_cached (4999 влучань у кеш із 5000 викликів).

4. Висновки

У ході роботи продемонстровано ключові переваги функціонального підходу в Python: усунення прихованих побічних ефектів, спрощення налагодження та тестування, безпека під час конкурентного доступу. Хоча копіювання при кожному оновленні створює додаткові витрати пам'яті (аллокація нового кортежу), оптимізації slots=True та мемоізація забезпечують високу швидкодію системи.